Guía paso a paso: tu primer pipeline seguro (taller DevSecOps)

Oct 27, 2026 · @Fabian

Antes de empezar

Necesitas cuatro cosas, todas gratuitas, y unos 10 minutos para tenerlas listas.

- **Cuenta de GitHub** en github.com.
- **Git**: comprueba que está instalado con `git --version`.
- **Node.js 20 o superior**: comprueba con `node --version` y `npm --version`.
- **Un editor de texto**, por ejemplo Visual Studio Code.

No hace falta instalar Python, Semgrep ni Gitleaks en tu computadora: esas herramientas corren dentro de GitHub Actions.

Paso 1: crear el repositorio y la app (10 min)

Al terminar este paso, `npm test` pasa en tu computadora y el código está subido a GitHub.

1. En GitHub, crea un repositorio nuevo (**New repository**) llamado `tienda-segura`. Márcalo como público y activa **Add a README file**.

2. Clónalo y entra en la carpeta (cambia `TU-USUARIO` por tu usuario):

   git clone https://github.com/TU-USUARIO/tienda-segura.git

   cd tienda-segura

3. Inicia el proyecto e instala Express:

   npm init -y

   npm install express

4. Abre `package.json` y reemplaza la línea de `"test"` dentro de `"scripts"` por esta:

   "test": "node --test"

5. Crea el archivo `.gitignore` con una sola línea, para no subir las librerías:

```
node_modules/
```

6. Crea el archivo `app.js`:

```
const express = require("express");
const app = express();
const total = (precio, cant, desc) =>
  precio * cant * (1 - desc);
app.get("/total", (req, res) => {
  const { p, c } = req.query;
  res.json({ total: total(p, c, 0) });
});
module.exports = { app, total };
```

7. Crea el archivo `app.test.js`:

```
const test = require("node:test");
const assert = require("node:assert");
const { total } = require("./app");
test("descuento del 50%", () => {
  assert.strictEqual(total(100, 2, 0.5), 100);
});
```

8. Ejecuta el test en tu computadora. Debe mostrar `pass 1` y `fail 0`:

```
npm test
```

9. Sube todo a GitHub, incluido `package-lock.json`, que el pipeline necesita:

```
git add .
```

git commit -m "App inicial con un test"

git push

Paso 2: el primer pipeline (10 min)

Un archivo YAML dentro del repositorio le dice a GitHub Actions qué ejecutar en cada pull request; en este paso solo corre los tests.

1. Crea una rama de trabajo:

   git checkout -b paso-2-pipeline

2. Crea la carpeta `.github/workflows` y dentro el archivo `pipeline.yml`. Usa espacios para la sangría, nunca tabulaciones:

   name: pipeline

   on: [pull_request]

   jobs:

     verificar:

       runs-on: ubuntu-latest

       steps:

         - uses: actions/checkout@v4

         - run: npm ci

         - run: npm test

3. Sube la rama:

   git add .

   git commit -m "Primer pipeline: solo tests"

   git push -u origin paso-2-pipeline

4. En GitHub, pulsa **Compare & pull request** y luego **Create pull request**.

5. Mira la sección de comprobaciones al final de la pull request, o la pestaña **Actions**. En uno o dos minutos debe aparecer una marca verde.

6. Pulsa **Merge pull request** y vuelve a `main` en tu computadora:

   git checkout main

   git pull

Paso 3: sumar las puertas de seguridad (15 min)

Cada puerta es una línea más en el mismo archivo, y juntas cubren tests, librerías, código y secretos.

1. Crea una rama nueva desde `main`:

   git checkout -b paso-3-puertas

2. Reemplaza todo el contenido de `.github/workflows/pipeline.yml` por esto:

   name: pipeline

   on: [pull_request]

   permissions:

    contents: read

    pull-requests: write

   jobs:

    verificar:

      runs-on: ubuntu-latest

      steps:

        - uses: actions/checkout@v4

          with: { fetch-depth: 0 }

        - run: npm ci

```
      - run: npm test
      - run: npm audit --audit-level=high
      - run: pip install semgrep
      - run: semgrep scan --config auto --error
      - uses: gitleaks/gitleaks-action@v2
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

3. Sube la rama y abre la pull request, igual que en el paso anterior:

   ```
   git add .
   git commit -m "Agregar puertas de seguridad"
   git push -u origin paso-3-puertas
   ```

4. Espera a que termine. Con el código original, todas las comprobaciones deben quedar en verde. Fusiona la pull request y actualiza `main` con `git checkout main` y `git pull`.

El bloque `permissions` no aparece en la diapositiva porque es una versión simplificada; lo necesita Gitleaks para escribir su resultado en la pull request.

Línea del pipeline	**Qué comprueba**	**Concepto**
`npm test`	Que la lógica sigue funcionando	Pruebas automáticas
`npm audit`	Vulnerabilidades conocidas en las librerías	SCA
`semgrep scan`	Patrones peligrosos en el código fuente	SAST
`gitleaks`	Claves y contraseñas en el código y el historial	Detección de secretos

Paso 4: plantar cuatro fallas (15 min)

Cada falla va en su propia rama y su propia pull request, así se ve con claridad qué puerta se pone en rojo. Antes de mirar el resultado, anota cuál crees que va a fallar.

Para cada falla repite este esquema, cambiando el nombre de la rama:

```
git checkout main

git pull

git checkout -b falla-secreto

# editar los archivos indicados abajo

git add .

git commit -m "Falla: secreto"

git push -u origin falla-secreto
```

Luego abre la pull request y espera el resultado.

Falla 1: un secreto en el código

Agrega esta línea al principio de `app.js`. Es un valor **inventado** con formato de token de GitHub; nunca uses una clave real.

```
const TOKEN = "ghp_aB3dE6gH9jK2mN5pQ8sT1vW4yZ7cF0hJ3kL6";
```

Falla 2: código inseguro

Agrega esta ruta en `app.js`, antes de la línea `module.exports`. Ejecuta como código lo que escribe el usuario en la dirección web.

```
app.get("/calcular", (req, res) => {

  res.json({ resultado: eval(req.query.expr) });

});
```

Falla 3: una librería vieja

Instala una versión de `lodash` con vulnerabilidades publicadas y sube los dos archivos que cambian, `package.json` y `package-lock.json`:

npm install lodash@4.17.15

Falla 4: un test roto

En `app.js`, cambia el signo de la fórmula del descuento:

precio * cant * (1 + desc);

Qué debería pasar

Rama	Paso en rojo	Mensaje esperado
`falla-secreto`	Gitleaks	Detecta un token de GitHub
`falla-codigo`	Semgrep	Marca la línea con `eval`
`falla-libreria`	`npm audit`	Informa vulnerabilidades altas en `lodash`
`falla-test`	`npm test`	El test esperaba 100 y recibió 300

Para leer el detalle de cada error, entra en la comprobación fallida y pulsa **Details**.

Paso 5: corregir y cerrar (10 min)

En tres de las cuatro ramas basta con empujar el arreglo y la pull request vuelve a verde sin abrir otra; la del secreto es distinta.

Falla 1: el secreto

Borrar la línea no alcanza: Gitleaks revisa los commits de la pull request y el token sigue en el historial, así que continúa en rojo. Cierra la pull request sin fusionar y borra la rama. Como nunca llegó a `main`, el secreto no se publicó. En un caso real, además, hay que revocar la clave y crear otra.

Falla 2: el código inseguro

Reemplaza la ruta con `eval` por una que no ejecute texto del usuario:

app.get("/calcular", (req, res) => {

  const resultado = Number(req.query.a) + Number(req.query.b);

```
  res.json({ resultado });
});
```

Falla 3: la librería vieja

Quita la dependencia que no usas y sube los dos archivos que cambian:

```
npm uninstall lodash
```

Falla 4: el test roto

Vuelve a poner el signo correcto en la fórmula:

```
precio * cant * (1 - desc);
```

En las tres ramas, haz commit y `git push`. Cuando la pull request quede en verde, fusiónala.

Lista de logros

- Un pipeline en verde con la app original
- Cuatro pull requests en rojo, cada una por un motivo distinto
- Tres corregidas y fusionadas, y la del secreto cerrada
- Puedes explicar qué puerta detectó cada falla

Para comentar en grupo: ¿qué falla habría llegado a producción sin el pipeline?

Retos extra

Si te sobra tiempo, estos tres retos acercan el taller a un proyecto real.

Reto 1: exigir el pipeline para fusionar

Hasta ahora un pipeline en rojo avisa, pero no impide fusionar. En GitHub, entra en **Settings**, luego **Branches** (o **Rules**), crea una regla para `main`, activa **Require status checks to pass before merging** y elige `verificar`. Prueba abrir una pull request con una falla: el botón de fusionar debe quedar bloqueado. En cuentas gratuitas, esta opción puede no estar disponible en repositorios privados.

Reto 2: escanear un contenedor y su configuración con Trivy

Crea un archivo `Dockerfile` en la raíz:

```
FROM node:20-alpine
```

```
WORKDIR /app

COPY package*.json ./

RUN npm ci --omit=dev

COPY . .

CMD ["node", "app.js"]
```

Agrega este paso al final de `pipeline.yml`:

```
    - run: docker run --rm -v "$PWD":/src aquasec/trivy:latest config --exit-code 1 --severity HIGH,CRITICAL /src
```

Normalmente Trivy avisa de que la imagen se ejecuta como administrador (root). Corrígelo agregando la línea `USER node` antes de `CMD`. En un proyecto real conviene fijar una versión concreta de Trivy en lugar de `latest`.

Reto 3: probar la app en ejecución con ZAP (DAST)

Es el más avanzado y puede requerir ajustes.

1. Agrega `app.listen(3000);` al final de `app.js` para que la app arranque un servidor.
2. En el pipeline, inicia la app en segundo plano con `node app.js &`.
3. Ejecuta el escaneo base de OWASP ZAP contra `http://localhost:3000`, siguiendo la documentación oficial de ZAP para GitHub Actions.

Problemas frecuentes

Los fallos más comunes se resuelven revisando la ruta del archivo, la sangría o los permisos.

Síntoma	**Causa probable**	**Solución**
El pipeline no se ejecuta	El archivo no está en `.github/workflows/` o no abriste una pull request	Revisa la ruta exacta y la extensión `.yml`; el pipeline solo corre en pull requests
`npm ci` falla	Falta `package-lock.json` en el repositorio	Haz commit de `package-lock.json`
Error de YAML	Sangría con tabulaciones o desalineada	Usa solo espacios y compara con el ejemplo

Síntoma	Causa probable	Solución
`npm test` no encuentra tests	El archivo no termina en `.test.js`	Renómbralo a `app.test.js`
Gitleaks pide una licencia	El repositorio pertenece a una organización	Usa un repositorio personal o instala el programa de Gitleaks y ejecútalo con `run`
Gitleaks falla por permisos	Falta el bloque `permissions`	Agrégalo como en el paso 3
`npm audit` falla con el código original	Apareció una vulnerabilidad nueva en una dependencia	Ejecuta `npm audit` en tu computadora, prueba `npm audit fix` o sube el umbral a `critical`
Semgrep no detecta `eval`	Las reglas cambian con el tiempo	Prueba `--config p/javascript` en lugar de `--config auto`

Los comandos, nombres de acciones y versiones de esta guía están escritos de memoria y no los ejecuté. Hazla completa una vez antes de la clase, porque las reglas y las bases de vulnerabilidades cambian.